

Содержание

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Мобильное приложение «CallShift» — перехват и переадресация входящих вызовов на Android

1. Общие сведения
2. Границы проекта (Scope)
3. Постановка задачи
4. Функциональные требования
5. Нефункциональные требования
6. Архитектура
7. Системные роли, разрешения и установка (критический раздел)
8. Пользовательский интерфейс
9. Модель данных
10. Алгоритмы
11. Выживаемость и борьба с ограничениями OEM
12. Обработка ошибок и краевые случаи
13. Логирование, наблюдаемость, приватность
14. Тестирование
15. Сборка, подписание, поставка APK
16. Риски
17. План работ, milestones и оценка
18. Критерии приёмки
19. Приложение А. Справочник MMI-кодов (GSM/3GPP TS 22.082, 27.007)
20. Приложение В. ADB-шпаргалка
21. Приложение С. Источники (нормативная база для разработчика)
22. Приложение D. Юридическая оговорка (обязательна в README)
23. Приложение E. Ограничения публичного Android SDK, выявленные при разработке референс-сборки

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Мобильное приложение «CallShift» — перехват и переадресация входящих вызовов на Android

Поле	Значение
Наименование изделия	CallShift (рабочее название)
Тип изделия	Нативное Android-приложение, распространяемое как APK (sideload)
Версия документа	1.1 (добавлено Приложение Е по итогам М0/М1)
Дата	24.09.2026
Статус	На согласование
Назначение	Личное использование, без публикации в Google Play
Целевая аудитория	1 разработчик Android (middle+/senior), срок 4–6 недель
Стек (рекомендация)	Kotlin 2.x, Jetpack (ViewModel, Room, DataStore, WorkManager), UI: XML Views в MVP → Jetpack Compose в МЗ

1. Общие сведения

1.1. Назначение документа

Документ определяет полный объём требований к приложению, которое **перехватывает входящий вызов на уровне Android Telecom Framework и перенаправляет его по заданному правилу** — на другой номер, на SIP-аккаунт или на оператора связи (сеть), а также уведомляет пользователя о факте вызова.

Документ является основанием для:

- оценки трудозатрат и составления плана работ;
- разработки архитектуры и кода;
- приёмки результата (см. раздел 18 «Критерии приёмки»).

1.2. Ключевое проектное решение, зафиксированное заказчиком

Параметр	Решение
Механика	Перехват и перенаправление звонков (CallScreeningService + ConnectionService), а не только MMI-коды оператора
Стек	Выбирает исполнитель (рекомендация — нативный Kotlin)
Масштаб	Личное использование, установка через adb/sideload, максимум возможностей , включая сценарии с ADB-грантами и root
Формат поставки	APK + исходный код + ТЗ + инструкции

1.3. Термины и сокращения

Термин	Определение
CF (Call Forwarding)	Переадресация вызова на стороне сети оператора (GSM/VoLTE supplementary service)
MMI	Man-Machine Interface — сервисные коды вида *21*<номер>#, исполняемые сетью
USSD	Unstructured Supplementary Service Data — сессия запрос/ответ с сетью
Screening	Процедура досмотровой фильтрации входящего вызова системой Android до показа пользователю
Role / Роль	Системная роль приложения (<code>RoleManager.ROLE_CALL_SCREENING</code> , <code>ROLE_DIALER</code>)
Self-managed Connection	Режим <code>ConnectionService</code> , при котором приложение само управляет аудиопотоком вызова (SIP/VoIP)
Leg (плеер/нога)	Одно плечо вызова: GSM-плеер или SIP-плеер
Bridging (мост)	Соединение двух плеер с передачей звука между ними
OEM	Производитель устройства (Xiaomi, Samsung, Huawei и т. д.) со своей оболочкой
APK / AAB	Форматы пакетов Android: установочный файл / пакет для маркетплейса
PII	Персональные данные (в данном проекте — номера телефонов, журнал вызовов)

2. Границы проекта (Scope)

2.1. В объёме работ (In scope)

1. Движок правил переадресации с приоритетами, условиями и расписанием.
2. Перехват входящих вызовов через `CallScreeningService` (режим «экран screening-приложения»).
3. Режим «полного телефона»: приложение как **default dialer** (`ROLE_DIALER`) с собственным `InCallService` — даёт максимальные полномочия в Telecom.
4. Четыре стратегии перенаправления (см. 3.3): сеть (MMI), callback-дозвон, SIP/VoIP, уведомление-эскалация.
5. Управление сетевой переадресацией через MMI/USSD с отображением статуса по каждой SIM.
6. Журнал событий переадресации (локальная БД), уведомления, быстрый тайл/виджет.
7. Инсталлятор-мастер: проверка и выдача ролей/разрешений через `adb` (скрипт) и через системные настройки (ручной путь).
8. Сборка, подписание и поставка APK; CI-конвейер; инструкция по установке на устройство.
9. Комплект документации: README, инструкция установки, инструкция ADB-настройки, known issues по OEM.

2.2. Вне объёма (Out of scope), если не заказано отдельно

- Публикация в Google Play / RuStore и прохождение их политик (раздел 7.4 описывает ограничения).
- Бэкенд, личный кабинет, биллинг, мультипользовательность.
- Запись разговоров в файл (в объёме только транзит аудио; запись — отдельная задача с отдельными юридическими рисками).

- Интеграция с конкретными CRM/1С/контакт-центрами.
- Поддержка iOS.
- Локализация на языки, кроме RU/EN.

2.3. Допущения и ограничения среды

№	Допущение
D1	Устройство заказчика — Android 9.0 (API 28) и выше, одна или две SIM, приложение используется на доверенном личном устройстве. Внимание: роль <code>CALL_SCREENING</code> и <code>RoleManager</code> доступны только с Android 10 (API 29); на Android 9 перехват возможен исключительно в профиле B (<code>ROLE_DIALER</code>), т. к. <code>TelecomManager.getDefaultCallScreeningApp()</code> — <code>@SystemApi</code>
D2	Пользователь готов выполнить одноразовую настройку через <code>adb</code> (выдача ролей и runtime-разрешений без диалогов)
D3	<code>root</code> не обязателен для базовой функциональности; <code>root</code> -ветки (аудио-мост, скрытые API) — опциональный модуль M4
D4	Реальная SIM-карта обязательна для тестирования: эмулятор Android не поддерживает входящие GSM-вызовы и MMI/USSD полноценно
D5	Сетевая переадресация (MMI) зависит от оператора и может быть недоступна на некоторых MVNO/тарифах

3. Постановка задачи

3.1. Проблема

Штатные средства Android дают пользователю только «всё или ничего»: системная переадресация спрятана в настройках телефона, требует ручного ввода MMI-кодов, не зависит от условий (время, номер, SIM, занятость), не ведёт журнала и не умеет комбинировать сценарии «отклонить + позвонить на другой номер» или «отправить в SIP».

3.2. Цель

Единое приложение, которое по declarative-правилам решает судьбу каждого входящего вызова **за время, не заметное пользователю**, и обеспечивает один из исходов:

1. пропустить вызов как обычно;
2. отклонить/заглушить и перенаправить на целевой номер (callback-лег);
3. включить/выключить сетевую переадресацию на стороне оператора (работает даже при выключенном телефоне);
4. принять вызов и транслировать его в SIP (мост GSM ↔ VoIP);
5. уведомить пользователя/внешнюю систему о вызове, не мешая разговору.

3.3. Стратегии перенаправления — обязательная таблица выбора

Исполнитель реализует стратегии как **сменные модули** за общим интерфейсом `ForwardStrategy`. Выбор стратегии — в правиле.

ID	Стратегия	Как работает	Что видит звонящий	Что видит владелец	Зависимости	Пр
S1	Сетевая CF (MMI)	Приложение набирает *21*<номер># / *61*... / *67*... через TelecomManager.placeCall без UI; переадресацию выполняет сеть оператора	Слышит гудки и попадает на целевой номер; виден реальный номер владельца	Телефон может быть выключен — переадресация всё равно работает	Поддержка оператором; только на уровне сети; нет гибких условий «по приложению»	М
S2	Screening + callback-дозвон	CallScreeningService отклоняет вызов (setDisallowCall(true) + setRejectCall(true)), затем приложение совершает исходящий звонок на целевой номер	Вызов сбрасывается (или уходит в «пропущенные»), затем ему перезванивает владелец — либо владелец звонит на целевой номер	Видит уведомление «вызов перехвачен → дозвон на X»	Роль ROLE_CALL_SCREENING ; разрешение CALL_PHONE ; не является «настоящей» переадресацией — это переадресация через приложение	М (о
S3	SIP/VoIP-мост	Входящий GSM-вызов принимается, параллельно поднимается SIP-плеер через self-managed ConnectionService ; аудио связывается мостом	Говорит с абонентом SIP (или слышит голос владельца через мост)	Может слушать/говорить через SIP-приложение	SIP-стек (PJSIP/linphone-sdk), MANAGE_OWN_CALLS , стабильный интернет	М
S4	Уведомление / эскалация	Вызов пропускается, но факт вызова и метаданные отправляются в Telegram/webhook/на второй телефон	Обычный звонок	Мгновенное уведомление с кнопками «перезвонить», «переадресовать», «заблокировать»	INTERNET , POST_NOTIFICATIONS	М

Обязательное требование к честности продукта. В UI и в документации приложение должно явно различать S1 («реальная сетевая переадресация») и S2 («перехват и дозвон на уровне приложения»). Пользователь должен понимать: S2 **не передаёт исходный номер звонящего** целевому абоненту по сети — GSM не позволяет подменить CLI сторонним приложением. Компенсационное решение — раздел 10.6 (передача номера через DTMF/голосовое приветствие/SIP-заголовок P-Asserted-Identity в S3).

3.4. Пользовательские сценарии (user stories)

ID	Сценарий	Ожидание
US-1	«Ночью все звонки — на второй телефон»	Расписание 23:00–07:00, цель +358... , стратегия S1 или S2
US-2	«Звонки с неизвестных номеров — отклонять и слать мне в Telegram»	Условие «не в контактах», S4 + блокировка
US-3	«Когда я в роуминге — переадресовать всё на SIP»	Условие по сети/роумингу, S3
US-4	«Если занято — на голосовую почту оператора»	S1 с кодом *67*<номер>#
US-5	«Звонки от начальника — всегда пропускать громко»	Whitelist с наивысшим приоритетом, обход всех правил

ID	Сценарий	Ожидание
US-6	«Одной кнопкой выключить всю переадресацию»	Тайл Quick Settings / виджет / жест в уведомлении
US-7	«Покажи, что реально включено у оператора»	USSD-запрос *#21# и разбор ответа (best-effort)
US-8	«Сработало правило — покажи почему»	Журнал с указанием ID правила, условий и таймингов

3.5. Функциональные профили (режимы работы приложения)

Критично для архитектуры: объём доступных API **сильно зависит от системной роли** приложения.

Возможность	Профиль А: только ROLE_CALL_SCREENING	Профиль В: ROLE_DIALER (default dialer)	Профиль С: root / system
setDisallowCall	□	□	□
setRejectCall	□ (только вместе с disallow)	□	□
setRejectedAsMissed	△ метод @hide в публичном SDK: вызывать reflection'ом и деградировать до обычного отбоя, если метода нет	△ то же	△ то же
setSilenceCall (API 29+)	△ только при наличии ANSWER_PHONE_CALLS (выдаётся через adb); вызывать reflection'ом	□ автоматически по роли	□
setSkipNotification	△ только для default dialer / carrier app	□	□
setSkipCallLog	□ игнорируется (только carrier/system)	□ игнорируется	□
Управление вызовом (answer/reject/hold/DTMF)	□	□ через InCallService	□
Чтение CallLog без runtime-диалога	□ (нужен грант)	□ неявно по роли	□
Исходящий звонок без диалога	△ нужен CALL_PHONE	□	□
Self-managed SIP-вызовы	□ (MANAGE_OWN_CALLS, normal)	□	□
Доступ к аудио вызова (VOICE_CALL)	□	△ частично при активном self-managed connection	□

Требование: приложение должно при старте определять свой профиль (PermissionProfileDetector) и **деградировать функциональность корректно**, показывая в UI, какие функции недоступны и какой командой это лечится. Профиль В — рекомендуемый для «максимума возможностей» (раздел 7.2).

4. Функциональные требования

Обозначения приоритета: **M** — must (обязательно для приёмки), **S** — should, **C** — could.

4.1. Правила переадресации

ID	Требование	Приоритет
FR-1.1	Пользователь может создавать, редактировать, удалять, дублировать и упорядочивать правила	M
FR-1.2	Каждое правило содержит: имя, флаг активности, приоритет (число, чем меньше — тем выше), условия, действие (стратегия + цель), расписание, срок действия	M
FR-1.3	Условия (логическое «И» внутри группы, «ИЛИ» между группами): номер/маска/E.164-диапазон, «в контактах» / «не в контактах», входящий/исходящий, конкретная SIM (PhoneAccountHandle), состояние линии (свободна/занята/не отвечает), тип сети (Wi-Fi/мобильная/роуминг), заряд батареи, время суток и дни недели, гео-зона (опционально, ACCESS_FINE_LOCATION)	M (geo — C)
FR-1.4	Действия: пропустить; отклонить; заглушить (silence); отклонить и отметить как пропущенный; S1 (MMI-включение/дозвон); S2 (reject + callback на цель); S3 (SIP-мост на аккаунт); S4 (уведомление/webhook); заблокировать навсегда	M
FR-1.5	Матч правил — по приоритету, первое совпавшее побеждает; если ни одно не совпало — применяется «политика по умолчанию» (настраивается)	M
FR-1.6	Правило может иметь «тест-режим»: прогон гипотетического номера без реального действия	S
FR-1.7	Импорт/экспорт правил в JSON (бэкап/перенос на новое устройство)	M
FR-1.8	Правила хранятся локально (Room), никаких облачных синхронизаций	M

4.2. Перехват входящих вызовов

ID	Требование	Приоритет
FR-2.1	Реализовать CallScreeningService с onScreenCall(Call.Details) ; ответ отдаётся методом respondToCall(Call.Details, CallResponse) строго тем же объектом Call.Details , который пришёл в колбэк (не строковым callId). Ответ формируется не более чем за 3000 мс (жёсткий системный таймаут ≈ 5 с, при превышении система пропускает вызов)	M
FR-2.2	Все тяжёлые операции (поиск в контактах, гео, сеть) выполнять асинхронно с предварительным кэшем; ответ в screening — только по кэшу/быстрым проверкам	M
FR-2.3	При недоступности данных (например, ContactsContract не выдан) правило должно интерпретироваться безопасно: пропустить вызов , а не заблокировать	M
FR-2.4	Обрабатывать вызовы без номера (скрытый/ограниченный CLI): Call.Details.getHandle() == null → отдельная ветка правила «анонимный вызов»	M
FR-2.5	Поддержка двух SIM: Call.Details.getPhoneAccountHandle() закрыт для сторонних приложений (@hide/@SystemApi), поэтому SIM определяется best-effort : reflection → ключи phoneAccount* в Call.Details.getExtras() → «не определена». Если SIM не определена, правило с селектором SIM трактуется как «любая SIM» (fail-open), а в журнал пишется sim=undefined. Полный доступ к SIM даёт только профиль B (ROLE_DIALER)	M
FR-2.6	Никогда не перехватывать экстренные номера (определять по Call.Details + локальному списку 112, 911, 100... + TelephonyManager/CarrierConfig): всегда пропускать	M
FR-2.7	Не обрабатывать вызовы от self-managed соединений (VoIP-звонки других приложений), если пользователь явно не включил это в настройках	S

ID	Требование	Приоритет
FR-2.8	При любом необработанном исключении в сервисе — ответ «пропустить» (<code>disallow=false</code>), плюс запись в журнал ошибок и <code>crash-лог</code>	M
FR-2.9	Сервис не должен держать процесс «вечно»: после <code>respondToCall</code> — освобождение ресурсов; фоновый <code>foreground-сервис</code> только на время активной фазы дозвона/моста	M

4.3. Стратегия S1 — сетевая переадресация (MMI/USSD)

ID	Требование	Приоритет
FR-3.1	Поддержка полного набора кодов для каждой SIM: безусловная (21), по занятости (67), по недоступности (62), по отсутствию ответа (61), «все условные» (002/004), «все» (04) — установка/снятие/запрос. Идентификатор SIM получать из <code>PhoneAccountHandle.id</code> (поле <code>@hide</code>) с fallback на разбор <code>toString()</code> — формат <code>[<pkg>] <id></code>	M
FR-3.2	Отправка кода — через публичную сигнатуру <code>TelecomManager.placeCall(Uri.fromParts("tel", "*21*<номер>#", null), extras)</code> (API 26+) без показа UI; выбор SIM — через <code>extras.putParcelable("android.telecom.extra.PHONE_ACCOUNT_HANDLE", handle)</code> ; приём ответа USSD — через <code>extras.putParcelable("android.telecom.extra.RESULT_RECEIVER", ResultReceiver)</code> и ключ <code>android.telecom.extra.USSD_RESPONSE</code> (обе константы <code>@hide</code> , используются литералы). Fallback — <code>Intent.ACTION_CALL</code>	M
FR-3.3	Нормализация номера цели в E.164 (<code>libphonenumber</code>) перед подстановкой в код; запрет на подстановку невалидных символов	M
FR-3.4	Получение ответа USSD-запроса: использовать <code>TelecomManager.placeCall</code> с <code>EXTRA_RESULT_RECEIVER/ResultReceiver</code> (API 28+) и/или <code>PhoneStateListener/TelephonyCallback</code> для разбора результата; при недоступности — фиксировать «статус неизвестен»	S
FR-3.5	Локальное «желаемое состояние» (desired state) каждой настройки CF хранится в БД и отображается в UI отдельно от «фактического состояния сети» (если его удалось прочесть)	M
FR-3.6	Таймаут операции 15 с, ретрай не более 2 раз, явная индикация результата в UI и уведомлением	M
FR-3.7	Поддержка «быстрых пресетов»: Всё → номер; Занято → номер; Не ответил (N сек) → номер; Снять всё	M
FR-3.8	Обработка ошибок оператора (не поддерживается, нет сети, роуминг-запрет) с человекочитаемым сообщением и кодом причины из USSD-ответа	S

4.4. Стратегия S2 — screening + callback-дозвон

ID	Требование	Приоритет
FR-4.1	Последовательность: <code>respondToCall(disallow=true, reject=true)</code> → ожидание завершения вызова (не более 1500 мс) → исходящий дозвон на цель	M
FR-4.2	Дозвон: <code>Intent.ACTION_CALL</code> (профиль B / при наличии <code>CALL_PHONE</code>) либо <code>TelecomManager.placeCall</code> ; номер цели — из правила	M
FR-4.3	Защита от петель: если целевой номер == номеру звонящего или == номеру владельца — правило отклоняется как некорректное ещё на этапе сохранения (валидация в UI)	M

ID	Требование	Приоритет
FR-4.4	Защита от «шторма»: не более N дозвонov за период T (по умолчанию N=3, T=10 мин), настраивается	M
FR-4.5	Режимы дозвона: «сразу», «с задержкой X сек», «только если экран включён», «спросить уведомлением» (кнопка «Перезвонить на цель»)	S
FR-4.6	Исходный номер звонящего сохраняется в журнал события и показывается в уведомлении; по возможности передаётся цели (см. 10.6)	M
FR-4.7	При неудаче дозвона — уведомление с причиной и повторной кнопкой	M

4.5. Стратегия S3 — SIP/VoIP-мост

ID	Требование	Приоритет
FR-5.1	Подключение ≥ 1 SIP-аккаунта: домен, пользователь, пароль, порт, транспорт (UDP/TCP/TLS), прокси; хранение пароля — в EncryptedSharedPreferences / Keystore	S
FR-5.2	Реализовать ConnectionService с self-managed соединениями (registerSelfManagedConnection, разрешение MANAGE_OWN_CALLS) для SIP-плеера	S
FR-5.3	Режимы: (а) «слепая переадресация» — GSM-вызов отклоняется, SIP-плеер звонит на цель; (б) «мост» — GSM-вызов удерживается/принимается, поднимается SIP-плеер, аудио связывается; (с) «конференция»	S
FR-5.4	Аудио-мост: захват через AudioRecord(MediaRecorder.AudioSource.VOICE_COMMUNICATION) + воспроизведение в SIP-плеер; обратное направление — симметрично; задержка ≤ 250 мс	S
FR-5.5	Явно предупреждать пользователя: на устройствах без root захват аудио GSM-плеера ограничен политиками Android 10+ (CAPTURE_AUDIO_OUTPUT — signature-разрешение); полный мост возможен в профиле C (root) или через удержание вызова в self-managed connection	M (как требование к документации и UI)
FR-5.6	Восстановление после обрыва интернета: ретрай регистрации SIP ≤ 3 раз, при неуспехе — fallback на S2/S4	S
FR-5.7	DTMF-транзит (RFC 2833 / SIP INFO)	C

4.6. Стратегия S4 — уведомление / эскалация

ID	Требование	Приоритет
FR-6.1	Мгновенное уведомление (high priority) с: именем/номером, SIM, сработавшим правилом, кнопками «Ответить», «Переадресовать на X», «Заблокировать», «Открыть журнал»	M
FR-6.2	Отправка события в Telegram (Bot API sendMessage), в произвольный webhook (HTTP POST JSON), по SMS на второй номер (опционально)	S
FR-6.3	Повторная отправка при отсутствии сети (WorkManager, backoff ≤ 15 мин, очередь ≤ 100 событий)	S
FR-6.4	Настройка «минимальная длительность звонка, о которой уведомлять» (фильтр флеш-звонков)	C

4.7. Журнал событий

ID	Требование	Приоритет
FR-7.1	Каждое срабатывание пишется в Room: время, номер (и хэш), направление, SIM, ID/имя правила, стратегия, цель, результат, длительность, причина ошибки, тайминги этапов (мс)	M
FR-7.2	Фильтры: по дате, правилу, результату, номеру; поиск; сортировка	M
FR-7.3	Экспорт в CSV/JSON; очистка по возрасту (автоудаление старше N дней, по умолчанию 90)	S
FR-7.4	Хранение — только локально; номер может быть замаскирован в UI по настройке «приватность»	M

4.8. Управление и интеграция с системой

ID	Требование	Приоритет
FR-8.1	Мастер первичной настройки (onboarding): шаги «разрешения → роль screening → роль dialer (опционально) → исключение из battery optimization → автозапуск (OEM) → проверка тест-звонком»	M
FR-8.2	Экран диагностики: таблица «что выдано / что нужно / команда adb, которая это выдаёт» + кнопка «скопировать команду»	M
FR-8.3	Quick Settings Tile «Переадресация: вкл/выкл» и переключение активного пресета	S
FR-8.4	Виджет 2x2 / 4x1 с текущим статусом и быстрым переключением	C
FR-8.5	Приём команд извне: broadcast-intent <code>fi.callshift.app.ACTION_SET_PRESET</code> с extra preset (для Tasker/Automate); защита — только с adb/системы (permission <code>android.permission.DUMP</code> или собственная signature-permission)	S
FR-8.6	Автозапуск после перезагрузки (<code>BOOT_COMPLETED</code>) — восстановление desired-state и foreground-сервиса, если он был включён	M

4.9. Настройки

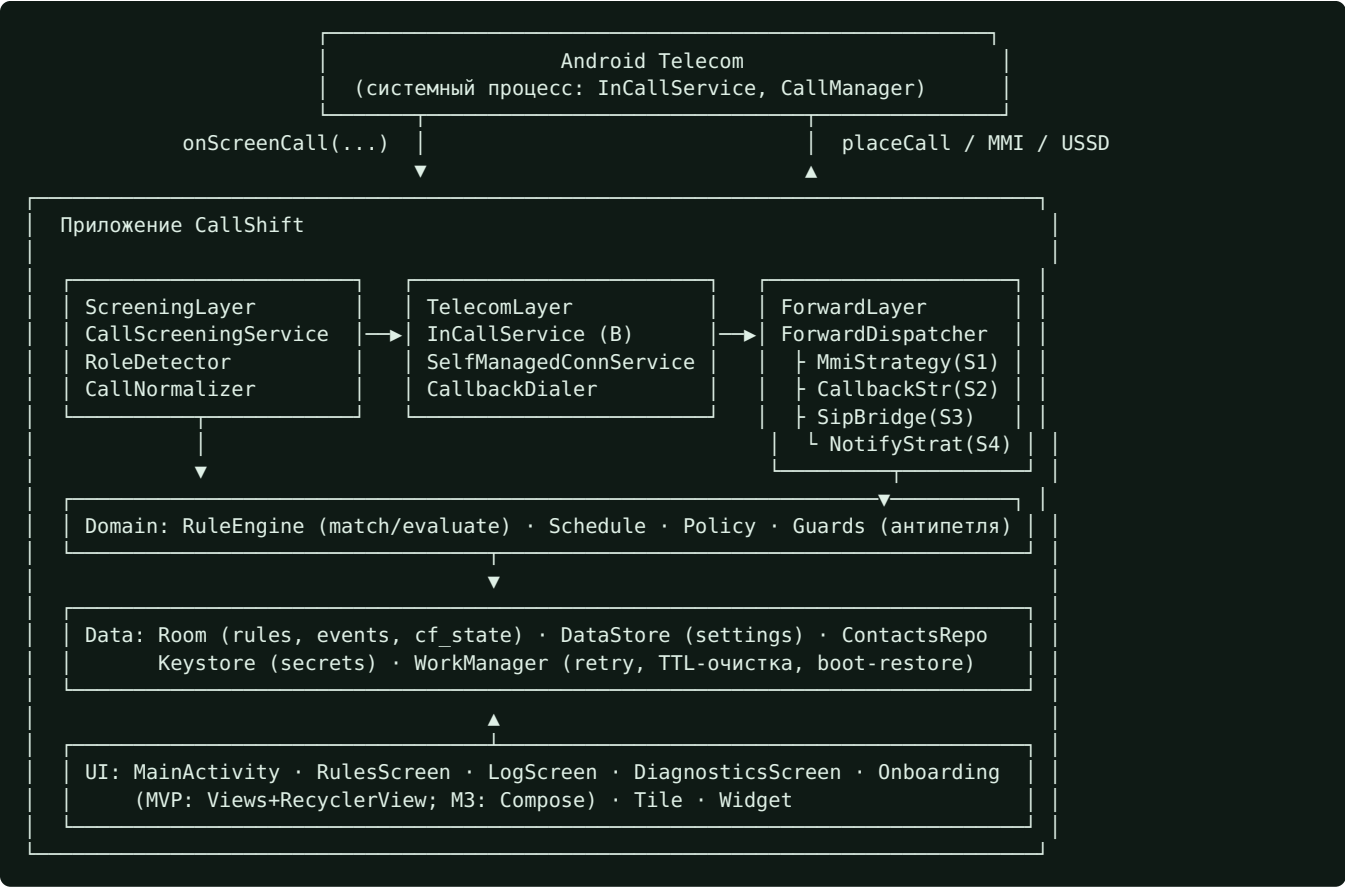
ID	Требование	Приоритет
FR-9.1	Глобальный «мастер-переключатель» (вся переадресация одним тумблером)	M
FR-9.2	Политика по умолчанию при отсутствии совпадений	M
FR-9.3	Уровень логирования (off/error/info/debug), экспорт логов в файл	M
FR-9.4	Приватность: маскировать номера, не хранить номера в логах, TTL журнала	M
FR-9.5	«Режим паники»: одна кнопка снимает все MMI-переадресации со всех SIM и выключает все правила	M
FR-9.6	Тёмная тема, языки RU/EN	S

5. Нефункциональные требования

ID	Категория	Требование
NFR-1	Производительность	Реакция screening-сервиса ≤ 3000 мс ($p95 \leq 300$ мс); холодный старт процесса по вызову screening ≤ 800 мс
NFR-2	Надёжность	0 (ноль) необработанных исключений в критическом пути вызова; при любом сбое — вызов пропускается (fail-open). Исключение: правила «блокировать спамеров» — fail-open тоже, но с уведомлением
NFR-3	Ресурсы	Потребление ОЗУ в простое ≤ 40 МБ; отсутствие wakeup-lock'ов в простое; $\leq 1\%$ батареи за 24 ч в режиме ожидания (без активных мостов)
NFR-4	Совместимость	Android 9 (API 28) — минимум, Android 16 (API 36) — целевая проверка; targetSdk = 35 ; ABI: arm64-v8a , armeabi-v7a , x86_64
NFR-5	ОЕМ	Проверено и документировано на: Xiaomi/MiUI-HyperOS, Samsung One UI, Huawei/HarmonyOS(EMUI), Google Pixel, OnePlus/Oppo. Для каждого — инструкция по автозапуску и whitelisting
NFR-6	Безопасность	Пароли SIP и токены — только в Android Keystore/EncryptedSharedPreferences; экспорт конфигурации — опционально с шифрованием (AES-GCM, пароль пользователя); никакие данные не покидают устройство, кроме явно настроенных webhook/Telegram
NFR-7	Приватность PII	Номера телефонов не отправляются в аналитику; crash-репорты (если включены) — с маскированием номеров; журнал — локально
NFR-8	Сопровождаемость	Покрытие unit-тестами домена (матч правил, нормализация номеров) $\geq 80\%$; detekt/ktlint в CI; версионирование SemVer; CHANGELOG.md
NFR-9	Размер	APK ≤ 15 МБ (без SIP-модуля); с SIP-модулем — ≤ 40 МБ
NFR-10	Доступность	Поддержка TalkBack, минимальный размер шрифта 14sp, контраст $\geq 4.5:1$

6. Архитектура

6.1. Общая схема



6.2. Слои и их ответственность

Слой	Ответственность	Запрещено
screening	Приём Call.Details, нормализация, быстрый (≤ 3 с) запрос решения у RuleEngine, формирование CallResponse	Любые сетевые вызовы, работа с БД на главном потоке, блокирующие IO
domain	Чистая логика: матчинг правил, расписания, guard'ы. Без Android-зависимостей (кроме моделей данных)	Знать о Telecom/Room
forward	Исполнение стратегий; каждая — отдельный класс за интерфейсом ForwardStrategy	Хранить состояние правил
telecom	Инкапсуляция системных API: placeCall, MMI, ConnectionService, InCallService	Логика правил
data	Room, DataStore, Keystore, WorkManager, репозитории	Прямые вызовы UI
ui	Отображение состояния, ввод правил, диагностика	Прямая работа с Telecom API (только через ViewModel → UseCase)

6.3. Ключевые интерфейсы (контракт для разработчика)

```
// domain/ForwardStrategy.kt
interface ForwardStrategy {
    val id: StrategyId           // S1 MMI | S2_CALLBACK | S3_SIP | S4_NOTIFY | BLOCK | PASS
    suspend fun execute(ctx: CallContext): ForwardResult
    fun isAvailable(profile: PermissionProfile): Boolean // деградация по профилю A/B/C
}

// domain/CallContext.kt
data class CallContext(
    val rawHandle: String?,      // может быть null (скрытый номер)
    val e164: String?,          // нормализованный
    val direction: Direction,   // INCOMING | OUTGOING
    val phoneAccount: PhoneAccountRef?, // SIM
    val isEmergency: Boolean,
    val isSelfManaged: Boolean,
    val timestampEpochMs: Long,
    val extras: Map<String, String>
)

// domain/RuleEngine.kt
interface RuleEngine {
    suspend fun evaluate(ctx: CallContext): Decision // < 3000 мс, fail-open
}
data class Decision(
    val verdict: Verdict,        // PASS | DISALLOW_REJECT | DISALLOW_AS_MISSED | SILENCE
    val strategy: StrategyId?,
    val target: String?,
    val ruleId: Long?,
    val reason: String,          // для журнала и отладки
    val screeningResponse: ScreeningFlags
)
```

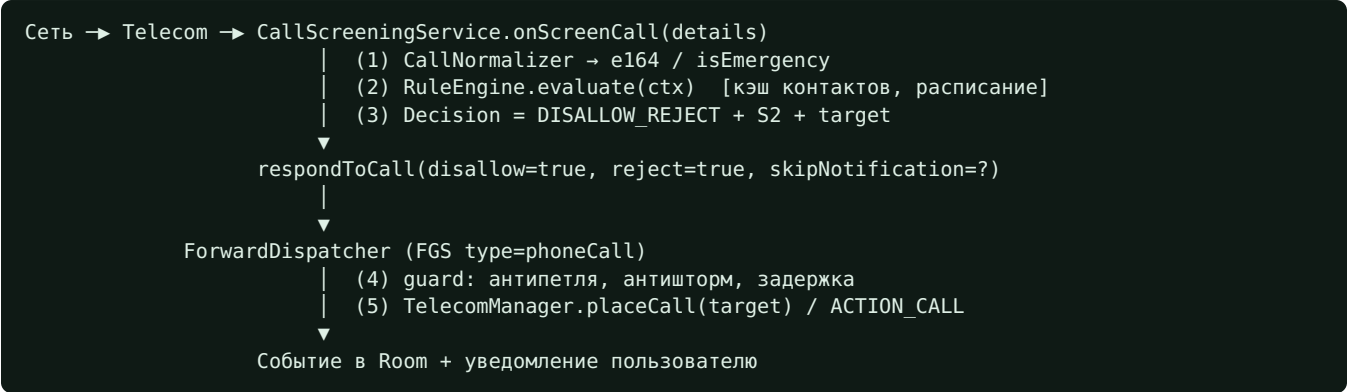
6.4. Поток и конкурентность

- Все suspend-функции — на `Dispatchers.Default` (логика) и `Dispatchers.IO` (БД/сеть).
- Screening-сервис живёт в основном процессе приложения; для минимизации cold start — **не** выносить в отдельный процесс (иначе каждый вызов платит за старт процесса).
- Долгие операции (дозвон, SIP-мост, webhook) — в `ForwardDispatcher`, запускаемом через foreground-сервис `type="phoneCall"` (Android 14+ требует явного типа FGS).
- Единая точка сериализации событий — `Actor`-подобная очередь (`Channel`), чтобы два одновременных входящих не устроили гонку дозвона.

6.5. Хранение

Сущность	Хранилище	Комментарий
rules	Room	версионирование схемы через Migration, <code>exportSchema = true</code>
events (журнал)	Room	индекс по timestamp, TTL-очистка WorkManager'ом
cf_state (desired state MMI по SIM)	Room	ключ = (simId, serviceCode)
settings	DataStore Preferences	типизированный wrapper
secrets (SIP-пароли, токены)	EncryptedSharedPreferences / Keystore	никогда не в Room и не в экспорте без пароля

6.6. Диаграмма последовательности (S2 — основной сценарий)



7. Системные роли, разрешения и установка (критический раздел)

7.1. Матрица разрешений

Разрешение	Уровень защиты	Зачем	Как получить (sideload)
READ_PHONE_STATE	dangerous	состояние линии, SIM	runtime-диалог или adb shell pm grant
READ_PHONE_NUMBERS (API 29+)	dangerous	собственный номер	runtime / adb
CALL_PHONE	dangerous	исходящий звонок (S2)	runtime / adb
ANSWER_PHONE_CALLS	dangerous	setSilenceCall, управление вызовом	только adb shell pm grant или роль dialer
READ_CALL_LOG / WRITE_CALL_LOG	dangerous	чтение/пометка вызовов	runtime / adb / неявно по роли dialer
READ_CONTACTS	dangerous	условие «в контактах»	runtime / adb
POST_NOTIFICATIONS (API 33+)	dangerous	уведомления	runtime / adb
MANAGE_OWN_CALLS	normal	self-managed ConnectionService (SIP)	автоматически
INTERNET, ACCESS_NETWORK_STATE	normal	webhook/Telegram, детект сети	автоматически
FOREGROUND_SERVICE, FOREGROUND_SERVICE_PHONE_CALL (API 34+)	normal	фоновая фаза звонка/моста	автоматически
RECEIVE_BOOT_COMPLETED	normal	восстановление после перезагрузки	автоматически
REQUEST_IGNORE_BATTERY_OPTIMIZATIONS	normal	выживаемость	runtime-диалог системы

Разрешение	Уровень защиты	Зачем	Как получить (sideload)
RECEIVE_SMS / SEND_SMS	dangerous	опционально: S4 через SMS	только если включён модуль
ACCESS_FINE_LOCATION	dangerous	опционально: гео-условия	только если включён модуль
CAPTURE_AUDIO_OUTPUT, MODIFY_AUDIO_ROUTING, READ_PRECISE_PHONE_STATE	signature/ privileged	полный аудио-мост, точное состояние	недоступно без root/system-подписи → профиль C

Обязательное требование FR-P1: manifest не должен запрашивать signature-разрешения в основной сборке — они вызывают отказ установки на некоторых устройствах и не имеют смысла без root. Выносить в отдельный root build variant.

7.2. Роли и команды настройки (профиль B — рекомендуемый)

Установка через adb (входит в поставку как scripts/setup_device.sh):

```
PKG=fi.callshift.app

# 1) Установка APK
adb install -r -t app-release.apk

# 2) Runtime-разрешения без диалогов
for p in READ_PHONE_STATE READ_PHONE_NUMBERS CALL_PHONE ANSWER_PHONE_CALLS \
    READ_CALL_LOG WRITE_CALL_LOG READ_CONTACTS POST_NOTIFICATIONS; do
    adb shell pm grant $PKG android.permission.$p || true
done

# 3) Роль screening-приложения (перехват входящих)
adb shell cmd role add-role-holder --user 0 android.app.role.CALL_SCREENING $PKG

# 4) Роль dialer (полный контроль Telecom; требует реализованных Dialer/InCallService)
adb shell cmd role add-role-holder --user 0 android.app.role.DIALER $PKG
# проверка:
adb shell cmd role get-role-holders --user 0 android.app.role.DIALER

# 5) Исключение из оптимизации батареи
adb shell dumpsys deviceidle whitelist +$PKG

# 6) (Xiaomi/OEM) автозапуск — только вручную в настройках оболочки, см. 11.5
```

Откат:

```
adb shell cmd role remove-role-holder --user 0 android.app.role.CALL_SCREENING $PKG
adb shell cmd role remove-role-holder --user 0 android.app.role.DIALER $PKG
adb shell pm revoke $PKG android.permission.ANSWER_PHONE_CALLS
```

Требование FR-P2: приложение должно уметь показать эти команды внутри UI (экран «Диагностика») и проверить результат после их выполнения (кнопка «Перепроверить»).

Требование FR-P3: при назначении ROLE_DIALER системный dialer заменяется — приложение **обязано** реализовывать минимально пригодный dialer (набор номера, экран входящего/активного вызова, ответ/отбой), иначе пользователь останется без телефона. Это требование — самая дорогая часть профиля B, оценка в разделе 17.

Требование FR-P4: при первом запуске без ролей приложение не должно «молча не работать»: показывать баннер «Перехват не активен» и вести в мастер настройки.

7.3. Ручной путь (без adb)

Настройки → Приложения → Приложения по умолчанию → Телефон / Определение номера и спам-фильтр (Call screening) → выбрать CallShift. Пункты меню различаются по OEM — документировать скриншотами для каждой оболочки (раздел 11.5).

7.4. Ограничения Google Play (справочно, т. к. проект — sideload)

Если в будущем потребуется публикация:

- приложение с CallScreeningService обязано объявить в manifest `<uses-permission android:name="android.permission.CALL_SCREENING"/>` и пройти проверку декларации;
- политика Play запрещает приложения, чья **основная** функция — перенаправление вызовов без согласия оператора/пользователя, а также «call forwarding» как основной сценарий для немодерируемого распространения;
- роль default dialer требует декларации `MANAGE_OWN_CALLS / ANSWER_PHONE_CALLS` и видео-демонстрации;
- публикация в Play **не входит** в объём данного ТЗ; при заказе — отдельная задача с аудитом политик.

8. Пользовательский интерфейс

8.1. Карта экранов

MainActivity (Bottom Navigation: Правила Статус Журнал Настройки)		
Правила		
└ RulesListScreen	–	список, drag-to-reorder приоритетов, свитчи активности
└ RuleEditScreen	–	мастер из 4 шагов: Условия → Действие → Расписание → Проверка
└ NumberPickerSheet	–	контакт / вручную / из журнала / маска
└ RuleTestScreen	–	«прогнать» номер, показать решение и причину
Статус		
└ CarrierForwardingScreen	–	по каждой SIM: 21/61/62/67/04, desired vs actual, пресеты
└ DiagnosticsScreen	–	таблица разрешений/ролей + adb-команды + кнопка «Перепроверить»
└ SipAccountsScreen (M3)	–	список аккаунтов, регистрация, статус
Журнал		
└ EventLogScreen	–	фильтры, поиск
└ EventDetailScreen	–	тайминги этапов, правило, причина
Настройки		
└ GeneralScreen	–	мастер-переключатель, политика по умолчанию, режим паники
└ PrivacyScreen	–	маскирование, TTL, экспорт/очистка
└ NotificationsScreen	–	каналы, webhook/Telegram
└ OnboardingWizard	–	пошаговая выдача ролей (показывается при первом запуске)

8.2. Требования к UI

ID	Требование
UI-1	Все деструктивные действия (снятие CF, удаление правила, режим паники) — с подтверждением и undo-тостом (5 с)
UI-2	На экране «Статус» крупно и однозначно: «Перехват активен / не активен», «Сетевая CF: включена на +358...», профиль полномочий A/B/C
UI-3	Любая недоступная из-за профиля функция — не скрывается, а показывается «заглушённой» с объяснением и командой для включения

ID	Требование
UI-4	Форма ввода номера: автоформатирование, валидация E.164, выбор страны по умолчанию (FI +358), подсказка о масках (+35840* , 8*)
UI-5	Material 3, светлая/тёмная тема, поддержка системного размера шрифта
UI-6	Экран журнала: группировка по дням, бейджи результата (успех/ошибка/пропущено), цвет — только с дублирующей иконкой (доступность)
UI-7	Время в таймингах события — в мс, с подсветкой > 1000 мс (диагностика деградации)

8.3. Состояния (state machine) главного экрана

NOT_CONFIGURED → PARTIAL (есть роль screening, нет разрешений) → ACTIVE → SUSPENDED (мастер-выключатель) → ERROR (роль отозвана системой)

Переход в ERROR обязан порождать уведомление «Переадресация не работает: роль отозвана» — частый сценарий после обновления OEM-оболочки.

9. Модель данных

9.1. rules

Поле	Тип	Комментарий
id	INTEGER PK	autoGenerate
name	TEXT	обязательно
enabled	INTEGER(bool)	
priority	INTEGER	0..999, уникальность не требуется
conditions_json	TEXT	сериализованная структура условий (см. 9.4)
strategy	TEXT	enum ForwardStrategy
target_e164	TEXT?	цель (номер/SIP URI), nullable для BLOCK/PASS
sim_selector	TEXT	`ANY
schedule_json	TEXT?	окна времени/дни/даты
valid_from / valid_to	INTEGER?	epoch ms
response_flags	INTEGER	битовая маска: DISALLOW REJECT AS_MISSED SILENCE SKIP_NOTIFICATION
created_at / updated_at	INTEGER	

9.2. events

Поле	Тип
id	INTEGER PK
ts	INTEGER (epoch ms)
direction	TEXT

Поле	Тип
number_e164 / number_hash	TEXT / TEXT
sim	TEXT
rule_id	INTEGER FK (nullable)
strategy	TEXT
target	TEXT
result	TEXT (`OK
error_code / error_msg	TEXT
timings_json	TEXT ({screening_ms, dial_ms, total_ms})

9.3. cf_state

Поле	Тип
sim_id	TEXT
service_code	TEXT (`21
desired_enabled	INTEGER
desired_target	TEXT?
desired_no_reply_seconds	INTEGER?
actual_enabled	INTEGER?
actual_target	TEXT?
last_sync_ts	INTEGER
last_error	TEXT?

9.4. JSON-схема условий (пример)

```
{
  "any_of": [
    {
      "all_of": [
        { "type": "number_match", "pattern": "+35840*" },
        { "type": "in_contacts", "value": false },
        { "type": "time_window", "days": [1,2,3,4,5], "from": "22:00", "to": "07:00" },
        { "type": "sim", "value": "SIM1" },
        { "type": "network", "value": "ROAMING" }
      ]
    },
    { "all_of": [ { "type": "anonymous", "value": true } ] }
  ]
}
```

Типы условий (MVP): number_match, number_in_list, in_contacts, anonymous, time_window, date_range, sim, line_state, network, battery_below. Расширение (M3): geo_fence, ssid, calendar_busy.

10. Алгоритмы

10.1. Нормализация номера

1. Получить `Call.Details.getHandle()` → `tel: URI` → строка.
2. `null`/пусто → `anonymous = true, e164 = null`.
3. `PhoneNumberUtil.parse(raw, defaultRegion)` → `format(E.164)`.
4. Дополнительно хранить `national` и `e164` для матчинга (оператор может отдать номер в национальном формате).
5. Маски: поддержка `*` (префикс/суффикс) и `?` (цифра); сравнение — по нормализованной строке; недопустимые маски отклоняются при сохранении.

10.2. Матчинг правил

```
вход: CallContext
1. if ctx.isEmergency → Decision(PASS, reason="emergency") // абсолютный приоритет
2. if !settings.masterEnabled → Decision(PASS, reason="master_off")
3. rules = repo.enabledRulesSortedByPriority() // кэш в памяти, инвалидация по Flow
4. for rule in rules:
    if rule.schedule.matches(now) && rule.conditions.matches(ctx) → return Decision(rule)
5. return Decision(defaultPolicy)
Ограничение: весь цикл ≤ 3000 мс; при TimeoutException → PASS + событие "engine_timeout"
```

Кэш контактов: предзагрузка множества нормализованных номеров в `HashSet` при старте процесса и по `ContentObserver`; при размере > 20 000 — переход на Room-запрос по индексу.

10.3. Антипетля и антишторм (обязательно)

Guard	Правило	Действие при срабатывании
Loop-self	<code>target == callerNumber</code>	отклонить правило, событие <code>guard_loop</code>
Loop-owner	<code>target == ownNumber(SIM)</code>	то же
Loop-recursion	входящий от <code>target</code> при активном правиле с тем же <code>target</code>	пропустить вызов (PASS), событие <code>guard_recursion</code>
Storm	> N дозвонov за T	PASS + уведомление «сработала защита от шторма»
Cooldown	тот же номер дважды за < 30 с	второе событие — PASS
Cost	номер цели в премиум-диапазоне (список по стране)	запрос подтверждения в настройках правила

10.4. Работа с MMI

```
buildMmi(service, target, noReplySec, mode) →
REGISTER: "*<code>*<target>#", для 61: "*61*<target>*<sec>#"
ERASE: "##<code>#"
INTERROGATE: "*<code>#"
ERASE_ALL: "##002#"
send: TelecomManager.placeCall(phoneAccountHandle, Uri.parse("tel:<mmi>"), extras)
    fallback: Intent.ACTION_CALL (профиль A без placeCall)
result: ResultReceiver → разбор текста USSD (best-effort, по локали оператора) → cf_state.actual_*
```

Требование: коды отправлять **именно на нужный SIM-account** (`PhoneAccountHandle` из `TelecomManager.getCallCapablePhoneAccounts()`), иначе на двухсимочных устройствах настройка уйдёт не на ту SIM.

10.5. Тайминги S2 (reject → callback)

Параметр	Значение по умолчанию	Обоснование
<code>reject_to_dial_delay_ms</code>	800	Telecom должен завершить обработку сброса, иначе новый вызов может быть отклонён как конфликтующий
<code>dial_timeout_ms</code>	20000	ожидание соединения
<code>max_retries</code>	1	
<code>screening_budget_ms</code>	3000	системный таймаут ~5 с

10.6. Передача исходного номера цели (ограничение платформы)

Вариант	Работает	Комментарий
Подмена CLI в GSM	❌	невозможно для стороннего приложения
DTMF после соединения	⚠️ только если цель — IVR/ATC, поддерживающая ввод	реализуемо в профиле B (<code>InCallService.playDtmfTone</code>)
Голосовое объявление номера (TTS в мост)	⚠️ требует аудио-моста (S3/root)	вариант для M4
SIP-заголовок P-Asserted-Identity / Diversion	❌ в S3	правильный способ для VoIP-инфраструктуры
Предварительное SMS/уведомление цели «вам перезвонит +X»	❌	MVP-компенсация, рекомендуется включить в FR-4.6

10.7. Аудио-мост (S3 / M4) — уровни возможности

Уровень	Условие	Что доступно
L0	Сток, профиль A/B	Только «слепая» переадресация без моста: GSM-вызов отклоняется, SIP-плеер звонит цели. Аудио не связывается
L1	Сток + self-managed connection удерживает GSM-плеер	Частичный мост: захват VOICE_COMMUNICATION, передача в SIP; качество зависит от OEM
L2	Root (<code>su</code>)	<code>AudioRecord(REMOTE_SUBMIX)</code> / <code>VOICE_CALL</code> при системных правах, полный двусторонний мост, возможна запись
L3	System-подпись / кастомная прошивка	Полный доступ к modem-audio, интеграция на уровне <code>ims</code> / <code>telephony</code>

Требование FR-A1: приложение обязано детектировать уровень (root-проверка `su`, проверка `granted signature-permissions`) и явно писать в UI, какой режим моста доступен. Запрещено обещать пользователю L2/L3 при фактическом L0.

11. Выживаемость и борьба с ограничениями OEM

11.1. Общие меры

- 1. `CallScreeningService` вызывается системой — он **не** подвержен обычному kill'у процесса, но холодный старт процесса платит за инициализацию → требование: инициализация ленивая, без тяжёлых библиотек в `Application.onCreate` (DI-граф — lazy, SIP-стек — только в МЗ и только по требованию).
- 2. Foreground service `type="phoneCall"` — только на время активной фазы (дозвон/мост), с корректным `stopSelf`.
- 3. `WorkManager` для отложенных задач (retry webhook, очистка журнала, периодическая сверка `cf_state` — не чаще 1 раза в 6 ч).
- 4. Исключение из battery optimization (`ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS` + `adb deviceidle whitelist`).
- 5. `BOOT_COMPLETED` → восстановление desired state, повторная проверка ролей.
- 6. Watchdog: `JobScheduler`-задача раз в 6 ч проверяет наличие ролей; при потере — уведомление.

11.2. Специфика OEM (таблица обязательна к документации в README)

Оболочка	Проблема	Мера
MIUI / HyperOS (Xiaomi)	«Автозапуск» по умолчанию выключен; агрессивная очистка; собственный «Блокировщик вызовов» может перехватить screening	Инструкция: Settings → Apps → Manage apps → CallShift → Autostart = ON; Battery saver = No restrictions; отключить конфликтующий системный фильтр
One UI (Samsung)	«Sleeping apps» / «Deep sleeping apps»; роль screening называется «Caller ID and spam protection»	Добавить в Unmonitored apps; adb-гранты работают штатно
EMUI / HarmonyOS (Huawei)	Ограничение фоновой активности; на устройствах без GMS роль dialer может вести себя иначе	Ручная выдача «Allow background activity»; проверка на конкретной модели обязательна
ColorOS/Realme UI, OxygenOS	App Cloner/Phone Manager убивает процессы	Whitelist в Phone Manager
Pixel / AOSP	Наиболее предсказуемо; роль screening выдаётся через «Caller ID & spam»	Эталонное устройство для CI-ручного тестирования

11.3. Конфликт с системными screening-приложениями

Требование FR-S1: на устройствах, где роль `CALL_SCREENING` уже занята (например, Google Phone / Mi Dialer), установка нашей роли **отзывает** предыдущую — приложение должно предупредить об этом в onboarding и предложить возврат (команда `adb shell cmd role remove-role-holder`).

Требование FR-S2: если пользователь держит системный спам-фильтр, наш движок правил должен учитывать, что часть вызовов может быть отфильтрована до нас — документировать, а не «бороться».

12. Обработка ошибок и краевые случаи

№	Ситуация	Ожидаемое поведение
E-01	Screening-роль отозвана системой/пользователем	Уведомление «перехват неактивен», статус ERROR, предложение команды восстановления
E-02	Не выдан READ_CONTACTS, правило его требует	Правило помечается «деградировано», условие трактуется как «не в контактах» с явной пометкой в журнале; вызов не блокируется ошибкой
E-03	Скрытый номер (handle == null)	Ветка «анонимный вызов»; если нет правила — политика по умолчанию
E-04	Экстренный номер	Всегда PASS, событие в журнал не пишется (приватность), кроме отладочного уровня
E-05	Второй входящий во время дозвона цели	Новый вызов обрабатывается независимо; дозвон прерывается по guard «cooldown»; в журнал — superseded
E-06	Входящий во время активного разговора (занято)	Учитывать условие line_state; MMI-код 67 — только через S1
E-07	Нет сети при отправке MMI	Retry ≤ 2, затем ошибка NO_NETWORK, desired state остаётся «ожидает применения», задача в WorkManager
E-08	Оператор не поддерживает код (USSD-ошибка)	Понятное сообщение + предложение альтернативы (S2)
E-09	Цель недоступна (дозвон не удался)	Уведомление, повтор по кнопке, событие DIAL_FAILED с причиной из InCallService/Call.State
E-10	Роуминг	По умолчанию — предупреждение о стоимости дозвона цели (guard Cost)
E-11	VoWiFi / VoLTE вызов	Поведение screening то же; MMI может быть недоступен → fallback S2
E-12	Вызов из другого приложения (self-managed: WhatsApp/Telegram)	По умолчанию не перехватывается (isSelfManaged), настраивается
E-13	Режим «Не беспокоить»	Screening продолжает работать (система вызывает сервис), но дозвон цели может быть подавлен DND → предупреждение в UI
E-14	Обновление приложения при активной роли	Роли сохраняются; миграции БД обязаны быть обратно совместимыми; проверка в CI
E-15	Смена SIM/eSIM, удаление SIM	cf_state помечается stale; правила с селектором HANDLE:<id> — перепривязка по PhoneAccountHandle.id, не по индексу; если id прочитать не удалось (ограничение платформы, FR-2.5) — правило трактуется как «любая SIM»
E-16	Пользователь удалил приложение, CF в сети осталась	В onboarding/README — обязательное предупреждение: перед удалением выполнить «Режим паники» (##002#); при удалении через ACTION_PACKAGE_FULLY_REMOVED приложение уже не может ничего сделать — документировать
E-17	Часы устройства сдвинуты / смена TZ	Расписание считается в локальной TZ с фиксацией ZoneId; при смене TZ — пересчёт
E-18	Переполнение журнала	Ротация: при > 50 000 записей удалять старейшие

13. Логирование, наблюдаемость, приватность

ID	Требование
LOG-1	Уровни: ERROR / WARN / INFO / DEBUG ; в релизе по умолчанию WARN ; переключается в настройках
LOG-2	В критическом пути screening — обязательные метки времени: t0_onScreenCall , t1_rule_matched , t2_respond , t3_dial_start , t4_dial_result
LOG-3	Номера в логах — маскируются (+35840***123) при уровне INFO и ниже; полный номер — только DEBUG и только локально
LOG-4	Экспорт логов — в файл в Documents/CallShift/ , с указанием версии приложения, Android API, модели, OEM, профиля полномочий
LOG-5	Crash-репортинг — выключен по умолчанию ; при включении (опционально, только для отладки разработчиком) — без PII
LOG-6	Никакой сторонней аналитики, рекламных SDK, трекеров. Это требование заказчика (личное использование)

14. Тестирование

14.1. Уровни

Уровень	Инструменты	Что покрываем	Порог
Unit	JUnit5, MockK, Truth	RuleEngine, нормализация номеров, маски, расписания, guard'ы, MMI-builder	≥ 80 % строк в domain
Интеграционные (локальные)	Robolectric	Room-миграции, репозитории, сериализация условий	все миграции
Приборные	AndroidX Test, Espresso/UI Automator	UI-поток создания правила, диагностика, экспорт	smoke-набор
Telecom-инструментальный	adb shell cmd telecom + TelecomManager -фейки, тестовый SIP-сервер (FreePBX в Docker)	placeCall, self-managed connection	M3
Ручные полевые	Реальная SIM, 2 устройства, тестовый номер	Единственный способ проверить screening и MMI	чек-лист 14.3
Нагрузочные/стресс	50 последовательных входящих (автодозвон со второго устройства)	отсутствие гонок, шторма, утечек	0 падений

Важно: эмулятор Android не поддерживает входящие GSM-вызовы и MMI/USSD; для CI допустимы только unit/Robolectric/Espresso. Полевые тесты — обязательная часть приёма.

14.2. Матрица устройств (минимум)

Устройство	Android	Оболочка	Профиль	Обязательность
Pixel 6/7/8	14-16	AOSP	B	обязательно

Устройство	Android	Оболочка	Профиль	Обязательность
Xiaomi (Redmi/Note)	12-14	MIUI/HyperOS	A/B	обязательно
Samsung Galaxy	13-15	One UI 6/7	A/B	обязательно
Устройство заказчика (основное)	факт	факт	B/C	обязательно
Huawei/Honor	10-12	EMUI	A	желательно

14.3. Чек-лист приёмочных полевых тестов (выдержка, полный — в [docs/TEST_CHECKLIST.md](#))

- ☐ Входящий с неизвестного номера отклоняется и уходит дозвон на цель (S2) — ≤ 3 с от звонка до начала дозвона
- ☐ Входящий из whitelist проходит штатно, без задержки и без уведомления
- ☐ Ночное расписание срабатывает в 23:00 и не срабатывает в 22:59 (проверка на реальном времени)
- ☐ Правило на SIM2 не влияет на SIM1
- ☐ Экстренный вызов 112 проходит всегда, даже при включённом правиле «блокировать всё»
- ☐ *21*<номер># реально включил переадресацию: проверка звонком с третьего номера **при выключенном телефоне**
- ☐ ##002# снял все переадресации
- ☐ После перезагрузки устройства правила работают без ручных действий
- ☐ После 24 ч в фоне (экран выключен, Doze) — перехват сработал
- ☐ Отзыв роли через системные настройки → приложение показывает ERROR и уведомление
- ☐ «Режим паники» за ≤ 10 с снимает все MMI и выключает все правила
- ☐ Журнал содержит все события с корректными таймингами
- ☐ Экспорт/импорт правил сохраняет их без потерь (включая расписания и маски)
- ☐ Нет петель: цель не звонит сама себе; при звонке с цели — вызов проходит
- ☐ Потребление батареи за 24 ч ожидания ≤ 1 % (по `dumpsys batterystats`)

14.4. Критерии качества

Метрика	Целевое значение
Доля корректно перехваченных вызовов (полевые, $n \geq 50$)	≥ 99 %
p95 latency screening	≤ 300 мс

Метрика	Целевое значение
Crash-free sessions	100 % (личное использование, допустимый минимум 99,5 %)
Ложные срабатывания (блокировка нужного вызова)	0

15. Сборка, подписание, поставка APK

15.1. Конфигурация Gradle (базовые требования)

Параметр	Значение
AGP	8.5+
Gradle	8.9+
Kotlin	2.0+
compileSdk / targetSdk	35
minSdk	28 (Android 9) — опционально 26, если нужен Android 8
versionCode / versionName	автоинкремент в CI; SemVer MAJOR.MINOR.PATCH
Build types	debug, release (R8 full mode, minifyEnabled=true, shrinkResources=true), variant root (доп. разрешения, L2-аудио)
ABI splits	arm64-v8a, armeabi-v7a, x86_64 + universal
Подпись	APK Signature Scheme v2 или v3 (v1/JAR не требуется при minSdk 28). Практика: при enableV3Signing=true apksigner поглощает блок v2, и apksigner verify показывает v2: false, v3: true — это штатное поведение. Поставочная сборка подписана v3 , проверка apksigner verify --min-sdk-version 28 = Verifies. Keystore не в репозитории: передаётся через CI secrets или локальный keystore.properties (в .gitignore), резервная копия — у заказчика (критерий A-9)
Репозиторий	Git (GitHub/GitLab), ветка main + feature-ветки, обязательный CI на PR

15.2. CI-конвейер (GitHub Actions, файл .github/workflows/android.yml)

1. checkout → setup-java (temurin 21) → кэш Gradle;
2. ./gradlew ktlint detekt (стиль и статический анализ) — блокирующий шаг;
3. ./gradlew testDebugUnitTest + jacocoTestReport — порог покрытия domain ≥ 80 %;
4. ./gradlew assembleRelease → артефакты app-arm64-v8a-release.apk, app-universal-release.apk;
5. подпись через apksigner (или Gradle signing config) + zipalign;
6. ./gradlew bundleRelease — опционально (AAB для будущей публикации);
7. публикация артефактов в Releases с автогенерацией changelog из conventional commits;
8. проверка apkanalyzer manifest print — контроль состава разрешений (регресс-защита от «случайно добавленного» разрешения).

15.3. Поставка заказчику

Артефакт	Формат
CallShift-vX.Y.Z-universal-release.apk	подписанный APK
CallShift-vX.Y.Z-arm64-v8a-release.apk	оптимизированный APK
Исходный код	Git-репозиторий с тегами
scripts/setup_device.sh	ADB-скрипт выдачи ролей и разрешений (раздел 7.2)
scripts/uninstall_cleanup.sh	снятие ролей + ##002#-инструкция + adb uninstall
README.md, INSTALL.md, OEM_NOTES.md, TEST_CHECKLIST.md, CHANGELOG.md	документация
Keystore + пароль	передаются заказчику отдельным защищённым каналом (обязательно: без него будущие обновления не встанут вверх)

15.4. Установка на устройство

```
adb install -r -t CallShift-v1.0.0-universal-release.apk
bash scripts/setup_device.sh
# проверка
adb shell cmd role get-role-holders --user 0 android.app.role.CALL_SCREENING
adb logcat -s CallShift:D
```

16. Риски

ID	Риск	Вероятность	Влияние	Митигция
R-1	Пользователь ждёт «настоящей» сетевой переадресации с сохранением номера звонящего, а получает S2	Высокая	Высокое	Явное разделение S1/S2 в UI и документации (3.3); пресет «Сетевая CF» как основной для сценария «телефон выключен»
R-2	OEM-оболочка отзывает роль / убивает процесс	Высокая	Высокое	Watchdog, уведомления, инструкции (11.2), тестирование на целевом устройстве заказчика в M0
R-3	Оператор не поддерживает MMI-коды (MVNO, виртуальные ATC, eSIM)	Средняя	Высокое	Детект ошибки USSD, fallback S2/S3, тест на SIM заказчика в M0 (спайк)
R-4	Профиль B (default dialer) требует полноценного dialer-UI — рост трудозатрат	Высокая	Среднее	M1 работает в профиле A; dialer-модуль — отдельный milestone M2/M3 с явной оценкой
R-5	Аудио-мост (S3) недостижим без root на стоковом устройстве	Высокая	Среднее	Честная детекция уровня L0-L3 (10.7); поставка «слепой» переадресации в SIP как основного сценария
R-6	Системный screening-таймаут (5 с) → вызов проходит, правило не сработало	Средняя	Высокое	Бюджет 3000 мс, кэши, fail-open, метрики таймингов в журнале

ID	Риск	Вероятность	Влияние	Митигция
R-7	Юридические риски: переадресация чужих вызовов, перехват, запись	Средняя	Высокое	Приложение для личного устройства и своих SIM; в README — дисклеймер о согласии сторон и законодательстве (в т. ч. GDPR/152-ФЗ при передаче данных в webhook)
R-8	Потеря keystore → невозможность обновления	Низкая	Высокое	Передача keystore заказчику, бэкап, документирование
R-9	Конфликт с системным спам-фильтром / другим screening-приложением	Средняя	Среднее	FR-S1/S2, диагностика в UI
R-10	«Режим паники» не снимет CF, если нет сети	Средняя	Среднее	Очередь команд в WorkManager + явное предупреждение пользователю

17. План работ, milestones и оценка

Оценка в человеко-днях (1 middle+/senior Android-разработчик). Включает код + тесты + документацию.

Milestone	Состав	Дни	Результат
M0. Спайк и фундамент (3–4 дня)	Каркас проекта, CI, подпись, проверка на целевом устройстве: выдача <code>ROLE_CALL_SCREENING</code> через adb, простейший <code>CallScreeningService</code> (отклонить все), отправка MMI <code>*21*.../##002#</code> , проверка USSD-ответа на SIM заказчика. Обязательный gate: если MMI не работает у оператора — фиксируем решение о приоритете S2/S3	4	apk-заготовка + отчёт о возможностях оператора/устройства
M1. Ядро: правила + S1 + S2 (10–12 дней)	RuleEngine, нормализация/маски, Room, UI списка и редактора правил, расписания, guard'ы, MMI-экран по SIM (desired/actual), callback-дозвон, журнал событий, уведомления, onboarding + диагностика, экспорт/импорт, режим паники, boot-restore	12	Рабочий APK профиля A: перехват + сетевая CF + callback
M2. Профиль B и выживаемость (7–9 дней)	Минимальный dialer (DialActivity, InCallService, экран входящего/активного вызова, answer/reject/DTMF), выдача <code>ROLE_DIALER</code> , tile/виджет, watchdog полей, OEM-инструкции, полный полевой прогон чек-листа	8	Максимальные полномочия, стабильность на 24 ч+
M3. S4 + SIP (S3, уровень L0/L1) (9–12 дней)	Telegram/webhook, очередь и retry, SIP-стек (PJSIP/linphone), self-managed <code>ConnectionService</code> , «слепая» SIP-переадресация, частичный аудио-мост L1, Compose-миграция UI (опционально)	11	SIP-переадресация, эскалация
M4. Root-модуль и аудио L2 (5–7 дней, опционально)	<code>root</code> build variant, <code>REMOTE_SUBMIX/VOICE_CALL</code> -захват, полный мост, TTS-объявление номера, DTMF-передача исходного номера	6	Полноценный мост на root-устройстве
M5. Полировка и приёмка (4–5 дней)	Оптимизация cold start, R8-правила, доступность, тёмная тема, финальные полевые тесты на 3 устройствах, документация, передача keystore	5	Релиз v1.0.0

Milestone	Состав	Дни	Результат
	Итого	46-52 дня	~9-10 недель при полной загрузке; при урезании до M1+M2 — **24 дня (~5 недель)**

17.1. Рекомендуемый минимальный объём для личного использования

$M0 + M1 + M2 = 24$ чел.-дня: перехват входящих по правилам, сетевая переадресация MMI, callback-дозвон, журнал, диагностика, tile, выживаемость на OEM. SIP (M3) и root-аудио (M4) — по решению заказчика.

17.2. Определение готовности (Definition of Done) для каждой задачи

- код + unit-тесты для доменной логики;
- отсутствие новых разрешений в manifest без явной отметки в T3/CHANGELOG;
- проверка на реальном устройстве (не эмулятор) для задач, связанных с Telecom;
- обновление README / OEM_NOTES, если затронуты инструкции;
- прохождение CI (lint, detekt, test, assembleRelease).

18. Критерии приёмки

№	Критерий	Как проверяем
A-1	APK устанавливается на целевое устройство и подписан (V2/V3); <code>adb install -r</code> поверх предыдущей версии работает	<code>apksigner verify --verbose</code> , установка
A-2	После <code>scripts/setup_device.sh</code> приложение показывает профиль полномочий A или B и статус «Перехват активен»	скриншот экрана диагностики
A-3	Все сценарии US-1...US-8 воспроизведены на реальной SIM	полевой протокол
A-4	Чек-лист 14.3 выполнен на 100 % для M1/M2	протокол тестирования
A-5	Покрытие <code>domain</code> unit-тестами ≥ 80 %, CI зелёный	отчёт JaCoCo
A-6	Нет сторонних SDK/трекеров; сетевые вызовы только на явно настроенные пользователем endpoint'ы	аудит зависимостей (<code>./gradlew dependencies</code>), mitmproxy -прогон
A-7	Потребление батареи в ожидании ≤ 1 % / 24 ч	<code>dumpsys batterystats</code>
A-8	Документация: README, INSTALL, OEM_NOTES, TEST_CHECKLIST, CHANGELOG — полные и актуальные	ревью
A-9	Keystore и пароли переданы заказчику, инструкция по резервному копированию приложена	подтверждение
A-10	«Режим паники» за ≤ 10 с снимает все переадресации на всех SIM	полевой тест с проверкой *#21#

19. Приложение А. Справочник MMI-кодов (GSM/3GPP TS 22.082, 27.007)

Сервис	Код	Установка	Снятие	Запрос статуса
Безусловная переадресация (CFU)	21	*21*<номер>#	##21#	*#21#
Переадресация по занятости (CFB)	67	*67*<номер>#	##67#	*#67#
По недоступности (CFNRc)	62	*62*<номер>#	##62#	*#62#
По отсутствию ответа (CFNRy)	61	*61*<номер>*<сек>#	##61#	*#61#
Все условные	002	—	##002#	—
Все переадресации	04	—	##04#	—
Регистрация/деактивация/стирание/запрос (общий синтаксис)	—	*<код>*<номер>#	#<код># / ##<код>#	*#<код>#
Ожидание вызова (Call Waiting)	43	*43#	#43#	*#43#
Определение номера (CLIP/CLIR)	30/31	*31# (скрыть)	#31#	*#31#

Примечания:

1. Времена для CFNRy: допустимые значения зависят от оператора (обычно 5, 10, 15, 20, 25, 30 с).
2. Некоторые операторы требуют формат *21*<номер># с номером в международном формате; другие принимают национальный. Требование FR-3.3: всегда E.164.
3. ##002# — «снять все условные переадресации»; рекомендуется в «Режиме паники» вместе с ##21#.
4. На VoLTE/eSIM часть кодов может обрабатываться иначе или возвращать ошибку — проверять в M0 на SIM заказчика.

20. Приложение В. ADB-шпаргалка

```
# роли
adb shell cmd role get-role-holders --user 0 android.app.role.CALL_SCREENING
adb shell cmd role add-role-holder --user 0 android.app.role.CALL_SCREENING fi.callshift.app
adb shell cmd role remove-role-holder --user 0 android.app.role.CALL_SCREENING fi.callshift.app
adb shell cmd role add-role-holder --user 0 android.app.role.DIALER fi.callshift.app

# разрешения
adb shell pm grant fi.callshift.app android.permission.ANSWER_PHONE_CALLS
adb shell dumpsys package fi.callshift.app | grep -A2 "runtime permissions"

# telecom / SIM
adb shell cmd telecom list-phone-accounts
adb shell cmd telecom set-sim-call-manager fi.callshift.app # для self-managed
adb shell dumpsys telecom | less

# батарея / фоновые ограничения
adb shell dumpsys deviceidle whitelist +fi.callshift.app
adb shell dumpsys deviceidle get deep

# логи
adb logcat -s CallShift:D Telecom:V TelephonyManager:V
adb shell dumpsys batterystats --charged fi.callshift.app

# проверка подписи
apksigner verify --verbose --print-certs CallShift-v1.0.0-universal-release.apk
```

21. Приложение С. Источники (нормативная база для разработчика)

1. Android Developers — `CallScreeningService`, `CallScreeningService.CallResponse.Builder` (ограничения `setRejectCall`/`setRejectedAsMissed`/`setSkipCallLog`/`setSkipNotification`).
2. Android Developers — `ConnectionService`, `Connection`, self-managed connections, `TelecomManager` (`placeCall`, `getCallCapablePhoneAccounts`, `EXTRA_RESULT_RECEIVER`).
3. Android Developers — `RoleManager` (`ROLE_CALL_SCREENING`, `ROLE_DIALER`), guide «Call screening app», «Build a dialer app».
4. Android Developers — «Background mic/camera restrictions» (Android 10+), «Foreground service types» (Android 14+), «Permissions for phone-related apps».
5. 3GPP TS 22.082 (Call Forwarding supplementary services), TS 27.007 (AT/MMI-команды), TS 24.080 (SS-протокол).
6. Google Play — «Call Screening / CALL_SCREENING declaration», «Permissions and Declarations policy» (справочно для раздела 7.4).
7. `libphonenumber` — нормализация E.164.
8. PJSIP / linphone-sdk — лицензионная совместимость (GPL/LGPL) при включении SIP-модуля: **обязательная проверка лицензии перед поставкой** (влияет на возможность закрытого распространения).

22. Приложение D. Юридическая оговорка (обязательна в README)

Приложение предназначено для управления **собственными** вызовами на **собственных** SIM-картах и устройствах. Переадресация вызовов третьих лиц без их согласия, перехват чужих переговоров и запись разговоров могут нарушать законодательство страны использования (в ЕС

— GDPR и национальные законы о тайне связи; в РФ — ст. 23 Конституции, 138 УК РФ, 152-ФЗ). Ответственность за использование несёт пользователь. Разработчик обязан включить этот текст в README и в экран первого запуска (одноразовое подтверждение).

Лист согласования

Роль	ФИО	Подпись	Дата
Заказчик			
Разработчик (исполнитель)			
QA			

Конец документа. Версия 1.1 — при изменении любого из разделов 3.3, 7, 10.7, 17 документ требует повторного согласования.

23. Приложение Е. Ограничения публичного Android SDK, выявленные при разработке референс-сборки

Раздел добавлен по итогам сборки M0/M1. Все пункты **обязательны к учёту** исполнителем: они меняют способ реализации, но не меняют функциональные требования.

№	Ограничение	Следствие	Обход (реализован в референс-сборке)
P-1	<code>CallScreeningService.respondToCall(Call.Details, CallResponse)</code> — первый параметр объект <code>Call.Details</code> , а не строковый <code>callId</code>	Наивная реализация «по документации из интернета» не компилируется	Сохраним <code>Call.Details</code> из коллбэка, а не <code>callId</code> , и передадим его же в <code>respondToCall</code>
P-2	<code>Call.Details.getPhoneAccountHandle()</code> — <code>@hide</code> / <code>@SystemApi</code>	Стороннее screening-приложение не видит, на какую SIM пришёл звонок	Reflection → <code>extras</code> → «не определено» (open). Полное решение — профиль (<code>ROLE_DIALER</code>)
P-3	<code>PhoneAccountHandle.id</code> — <code>@hide</code>	Нельзя напрямую сопоставить SIM и правило	Reflection к полю <code>id</code> , fallback — <code>toString()</code> (<code>[pkg] id</code>)
P-4	<code>CallResponse.Builder.setRejectedAsMissed()</code> и <code>setSilenceCall()</code> отсутствуют/скрыты в части версий SDK	Нельзя полагаться на прямые вызовы	Reflection <code>invokeIfAvailable(..)</code> с деградацией с записью в журнал
P-5	<code>Call.Details.PROPERTY_EMERGENCY_CALLBACK</code> — <code>@hide</code>	Прямая проверка недоступна	Чтение константы через reflection, базовая проверка — нормализация списка коротких номеров (FR-2.6)
P-6	Публичная сигнатура <code>TelecomManager.placeCall(Uri, Bundle)</code> — без отдельного параметра <code>PhoneAccountHandle</code>	Выбор SIM «в лоб» не компилируется	<code>EXTRA_PHONE_ACCOUNT_HANDLE</code> в <code>Bundle</code> ("android.telecom.extra.PHONE_ACCOUNT_HANDLE")
P-7	<code>TelecomManager.EXTRA_RESULT_RECEIVER</code> и <code>EXTRA_USSD_RESPONSE</code> — <code>@hide</code>	Приём ответа USSD через константы недоступен	Строковые литералы ключей; <code>onReceiveResult</code> (API 28+, требуется <code>ANSWER_PHONE_CALLS</code>)
P-8	<code>TelecomManager.getDefaultCallScreeningApp()</code> — <code>@SystemApi</code>	На Android 9 (API 28) проверить роль нельзя	<code>RoleManager</code> на API 29+; на API 28 — профиль B (см. D1)

№	Ограничение	Следствие	Обход (реализован в референс-приложении)
P-9	<code>PhoneNumberUtil.isEmergencyNumber()</code> отсутствует в публичном API <code>libphonenumber</code>	Проверка экстренных номеров «библиотечным» способом невозможна	Локальный набор коротких номеров 112 / 911 (FR-2.6); в M2 — сверка с базой
P-10	Kotlin-тесты с не-ASCII именами ломают сборку на JVM с не-UTF-8 <code>sun.jnu.encoding</code> (<code>InvalidPathException</code>)	CI падает на этапе <code>compileUnitTestKotlin</code>	Имена тестов — ASCII; в <code>gradle.properties</code> заданы <code>systemProp.sun.jnu.encoding=UTF-8</code> и <code>systemProp.file.encoding=UTF-8</code>
P-11	<code>setSkipCallLog()</code> игнорируется системой для сторонних приложений; вызовы всё равно пишутся в <code>CallLog</code> с типом <code>BLOCKED_TYPE</code>	«Скрыть след» в системном журнале нельзя	Документируем в UI; собственные события (FR-7.1) — источник правды

Практический вывод для заказчика. Пункты P-2, P-3, P-5 и P-11 означают, что «максимум возможностей» на стоковом устройстве достигается только в профиле B (`ROLE_DIALER`, T3 п. 3.5 и FR-P3) либо в профиле C (`root`). В профиле A приложение корректно перехватывает и отклоняет вызовы, выполняет сетевую переадресацию MMI и `callback`-дозвон, но не различает SIM и не может скрыть запись в системном журнале.